

Sistemas Operacionais

- Tipos de Sistemas Operacionais
- Funcionalidades
- Arquitetura:
 - Monolíticos
 - Em camadas
 - Micro-núcleo
 - Máquinas virtuais



Windows



Mac OS



Linux



Chrome OS



Android



iOS



Unix

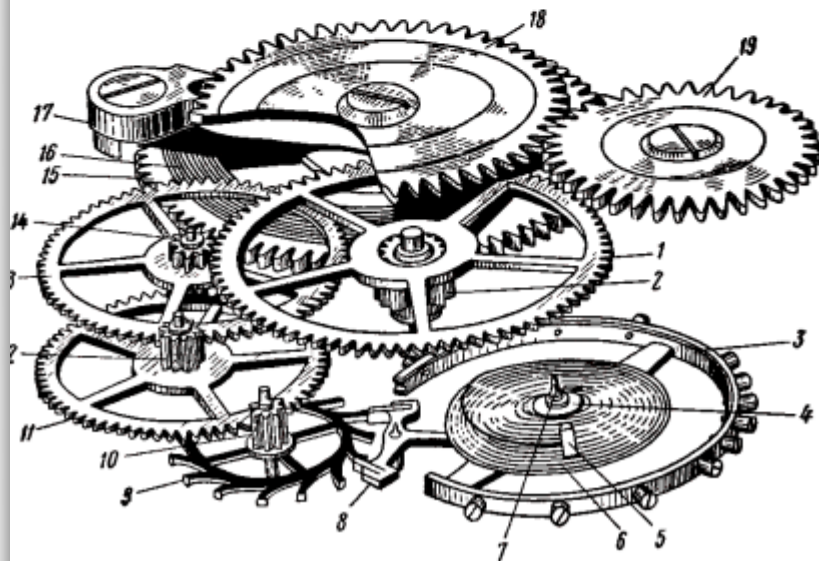


BSD

SISTEMAS OPERACIONAIS: CONCEITOS E MECANISMOS

PROF. CARLOS A. MAZIERO

DINF - UFPR

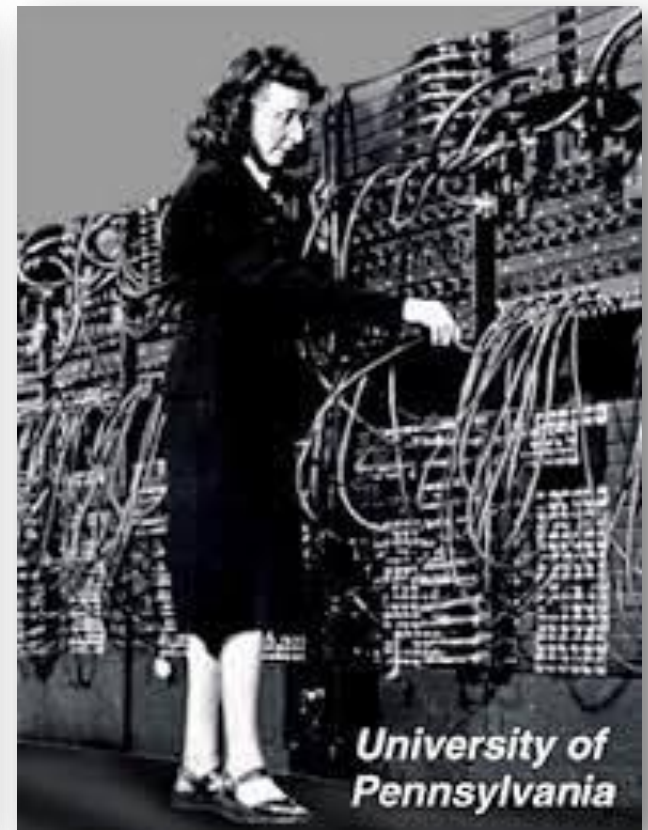
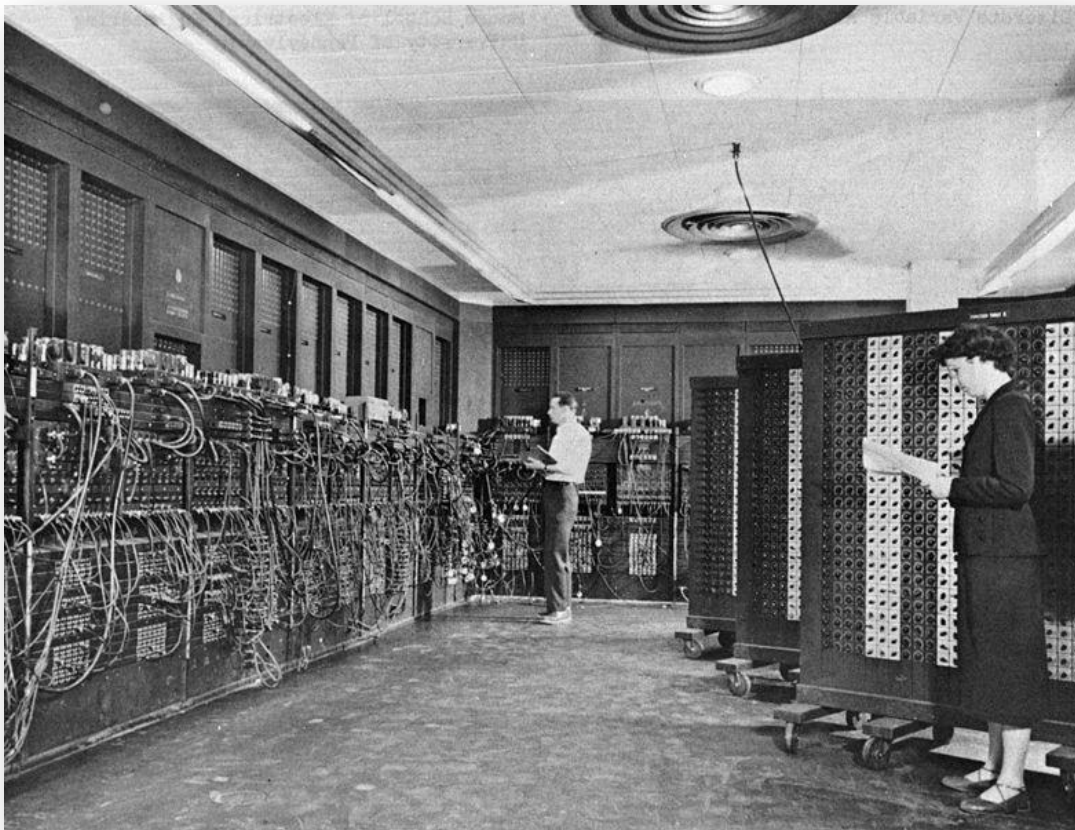


Este livro está disponível sob a licença [Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported](https://creativecommons.org/licenses/by-nc-sa/3.0/).

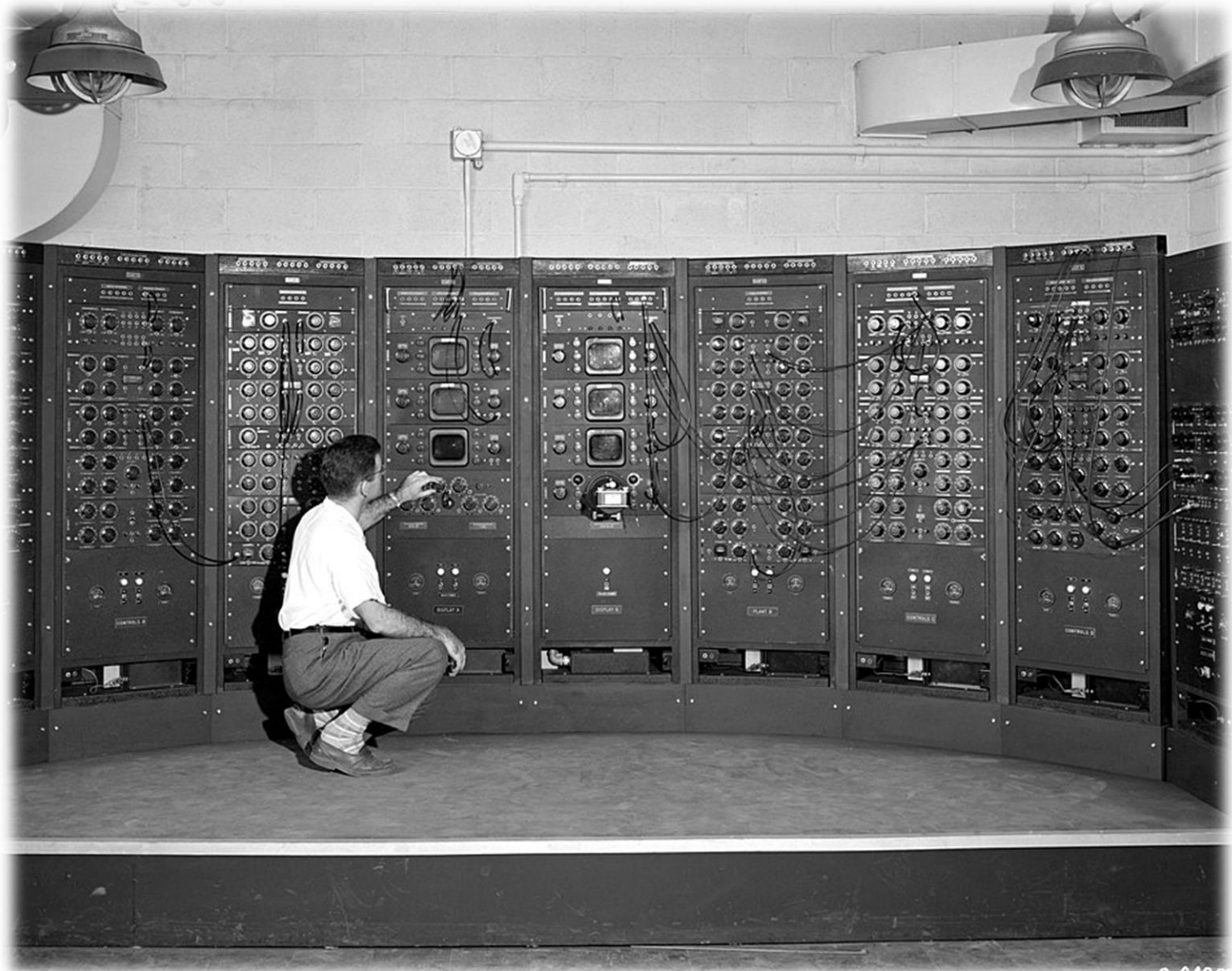
<https://wiki.inf.ufpr.br/maziero/lib/exe/fetch.php?media=socm:socm-livro.pdf>

Historia do SO

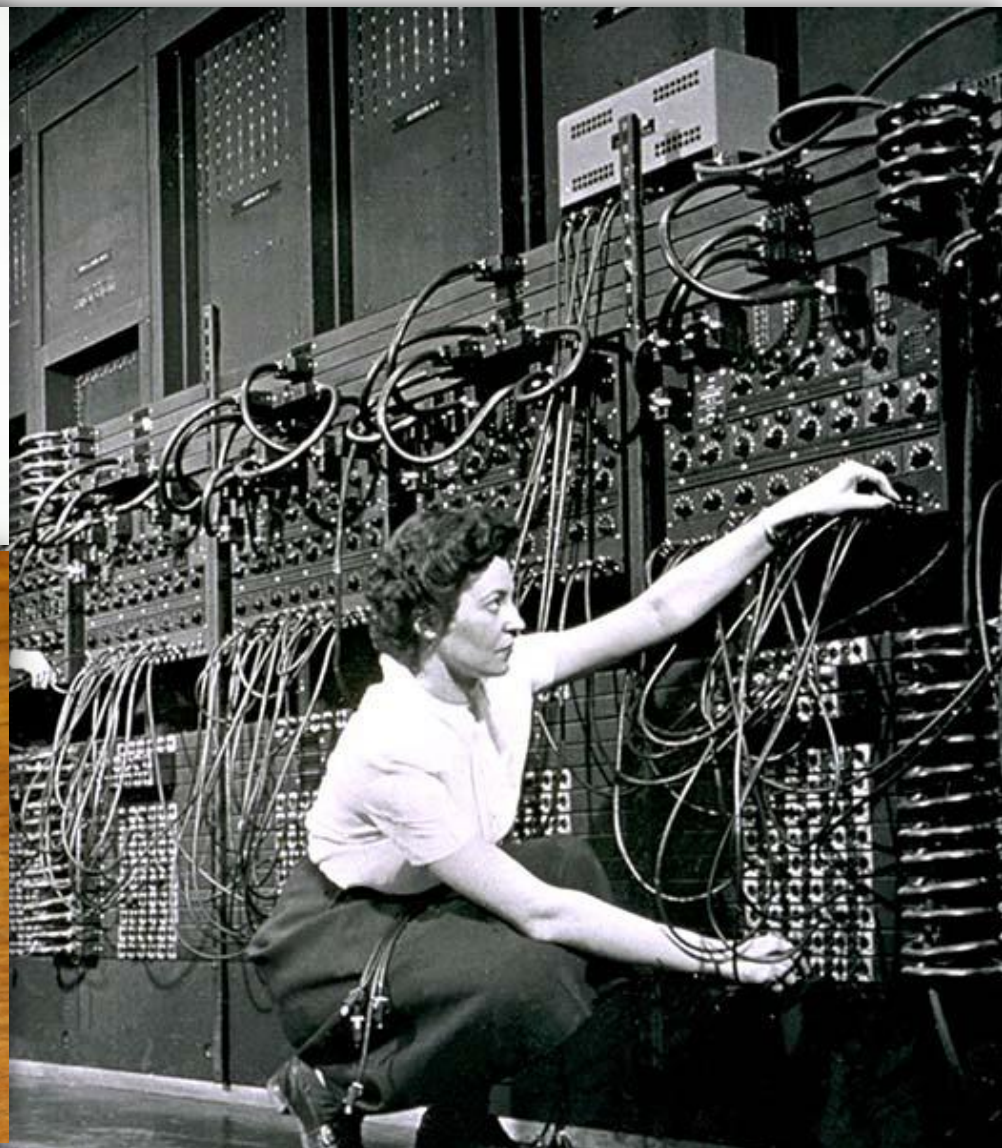
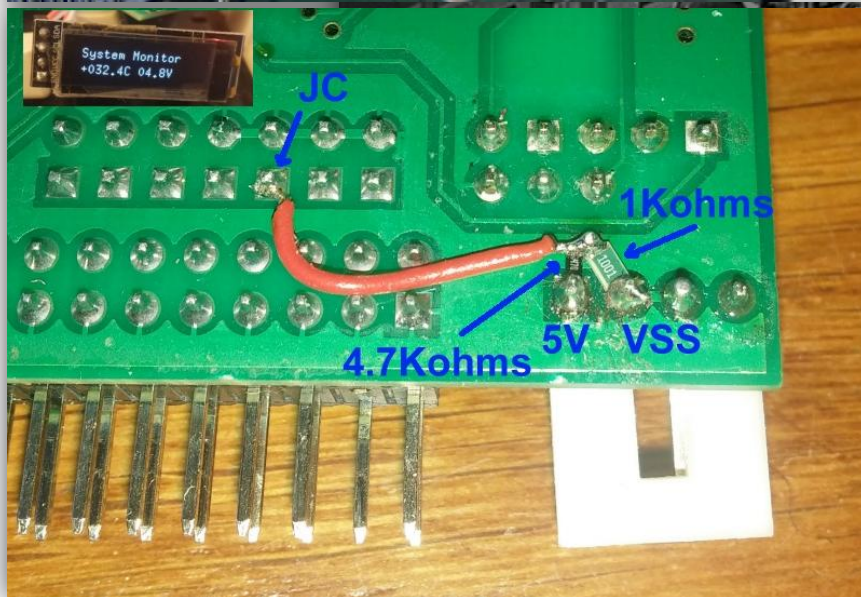
Os primeiros sistemas de computação, no final dos anos 1940, não possuíam sistema operacional: as aplicações eram executadas diretamente sobre o hardware.



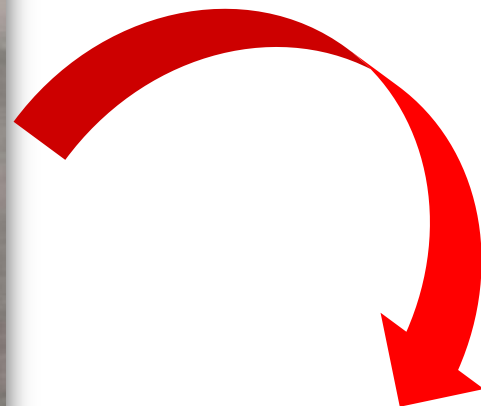
Historia do SO



Historia do SO



Historia do SO



Historia do SO

- **Anos 40:**
 - Cada programa executava sozinho e tinha total controle do computador (monotarefa).
 - A carga do programa em memória, a varredura dos periféricos de entrada para busca de dados, a computação propriamente dita e o envio dos resultados para os periférico de saída, byte a byte, tudo devia ser programado detalhadamente pelo desenvolvedor da aplicação. Ex. ENIAC
- **Anos 50:**
 - Os sistemas de computação fornecem “bibliotecas de sistema” (***system libraries***) que encapsulam o acesso aos periféricos, para facilitar a programação de aplicações.
 - Algumas vezes um programa “**monitor**” (system monitor) auxilia a carga e descarga de aplicações e/ou dados entre a memória e periféricos (geralmente leitoras de cartão perfurado, fitas magnéticas e impressoras de caracteres).

Historia do SO

- **1961:**
 - O grupo do pesquisador **Fernando Corbató**, do MIT, anuncia o desenvolvimento do CTSS – **Compatible Time-Sharing System**, o primeiro sistema operacional com compartilhamento de tempo.
- **1965:**
 - A IBM lança o OS/360, um sistema operacional avançado, com compartilhamento de tempo e excelente suporte a discos.
- **1965:** (surgiu e lei de Moore)
 - Um projeto conjunto entre **MIT, GE e Bell Labs** define o sistema operacional **Multics**, cujas ideias inovadoras irão influenciar novos sistemas durante décadas.
- **1969:**
 - **Ken Thompson e Dennis Ritchie**, pesquisadores dos Bell Labs, criam a primeira versão do UNIX.
- **1981:**
 - A Microsoft lança o MS-DOS, um sistema operacional comprado da empresa **Seattle Computer Products** em 1980.

Historia do SO

- **1984:**
 - A Apple lança o sistema operacional **Mac OS** 1.0 para os computadores da linha Macintosh, o primeiro a ter uma interface gráfica totalmente incorporada ao sistema.
- **1985:**
 - Primeira tentativa da Microsoft no campo dos sistemas operacionais com interface gráfica, através do **MS-Windows** 1.0.
- **1987:**
 - Andrew Tanenbaum, um professor de computação holandês, desenvolve um sistema operacional didático simplificado, mas respeitando a API do UNIX, que foi batizado como **Minix**.
- **1987:**
 - IBM e Microsoft apresentam a primeira versão do **OS/2**, um sistema multitarefa destinado a substituir o MS-DOS e o Windows. Mais tarde, as duas empresas rompem a parceria; a IBM continua no OS/2 e a Microsoft investe no ambiente Windows.

Historia do SO

- **1991:**
 - **Linus Torvalds**, um estudante de graduação finlandês, inicia o desenvolvimento do **Linux**, lançando na rede Usenet o núcleo 0.01, logo abraçado por centenas de programadores ao redor do mundo.
- **1993:**
 - A Microsoft lança o Windows NT, o primeiro sistema 32 bits da empresa, que contava com uma arquitetura interna inovadora.
- **1993:**
 - Lançamento dos UNIX de código aberto **FreeBSD** e **NetBSD**.
- **1993:**
 - A Apple lança o Newton OS, considerado o primeiro sistema operacional móvel, com gestão de energia e suporte para tela de toque.
- **1995:**
 - Microsoft lança o **Windows 95**.

Historia do SO

- **1999:**
 - A empresa **VMWare** lança um ambiente de virtualização para sistemas operacionais de mercado.
- **2001:**
 - A Apple lança o MacOS X, um sistema operacional com arquitetura distinta de suas versões anteriores, derivada da família **UNIX BSD**.
- **2005:**
 - Lançado o Minix 3, um sistema operacional micro-núcleo para aplicações embarcadas. O **Minix 3** faz parte do firmware dos processadores Intel mais recentes.

Historia do SO

- **2007:**
 - Lançamento do iPhone e seu sistema operacional **iOS**, derivado do sistema operacional Darwin (Darwin é um Kernel de código aberto lançado pela Apple Inc. em 2000).
- **2007:**
 - Lançamento do **Android**, um sistema operacional baseado no núcleo Linux para dispositivos móveis.
- **2010:**
 - Microsoft lança o **Windows Phone**, SO para celulares pela Microsoft.
- **2015:**
 - Microsoft lança o **Windows 10** e em 2021 lança o **Windows 11**.

O que é um Sistemas Operacional?



Conceito de Sistemas Operacionais



O sistema operacional é uma camada de software que opera entre o hardware e os programas aplicativos voltados ao usuário final.



Conceito de Sistemas Operacionais

Os circuitos são complexos, acessados através de interfaces de baixo nível e muitas vezes suas características e seu comportamento dependem da tecnologia usada em sua construção.

- **Exemplo:** A forma de acessar um disco rígido IDE de um disco SSD.

Conceito de Sistemas Operacionais

Linguagem *Assembly*

```
.INCLUDE "M32DEF.inc" ;inclui regs
.EQU OP1VAR = 0x109   ;reserva end.
.EQU OP2VAR = 0x10A   ;dados p variável
.EQU RESVAR = 0x10B

.ORG 0                ;indica início do código
LDI    R16, 5         ;carrega variáveis
STS    OP1VAR, R16
LDI    R16, 12
STS    OP2VAR, R16
    ...
LDS    R16, OP1        ;carrega R16 com OP1
LDS    R17, OP2        ;carrega R17 com OP2
ADD    R16, R17        ;soma, guarda em R16
STS    RESVAR, R16    ;devolve res
                        ;para mem. de dados
    ...
```

Linguagem C

```
#include <avr/io.h>    //inclui regs

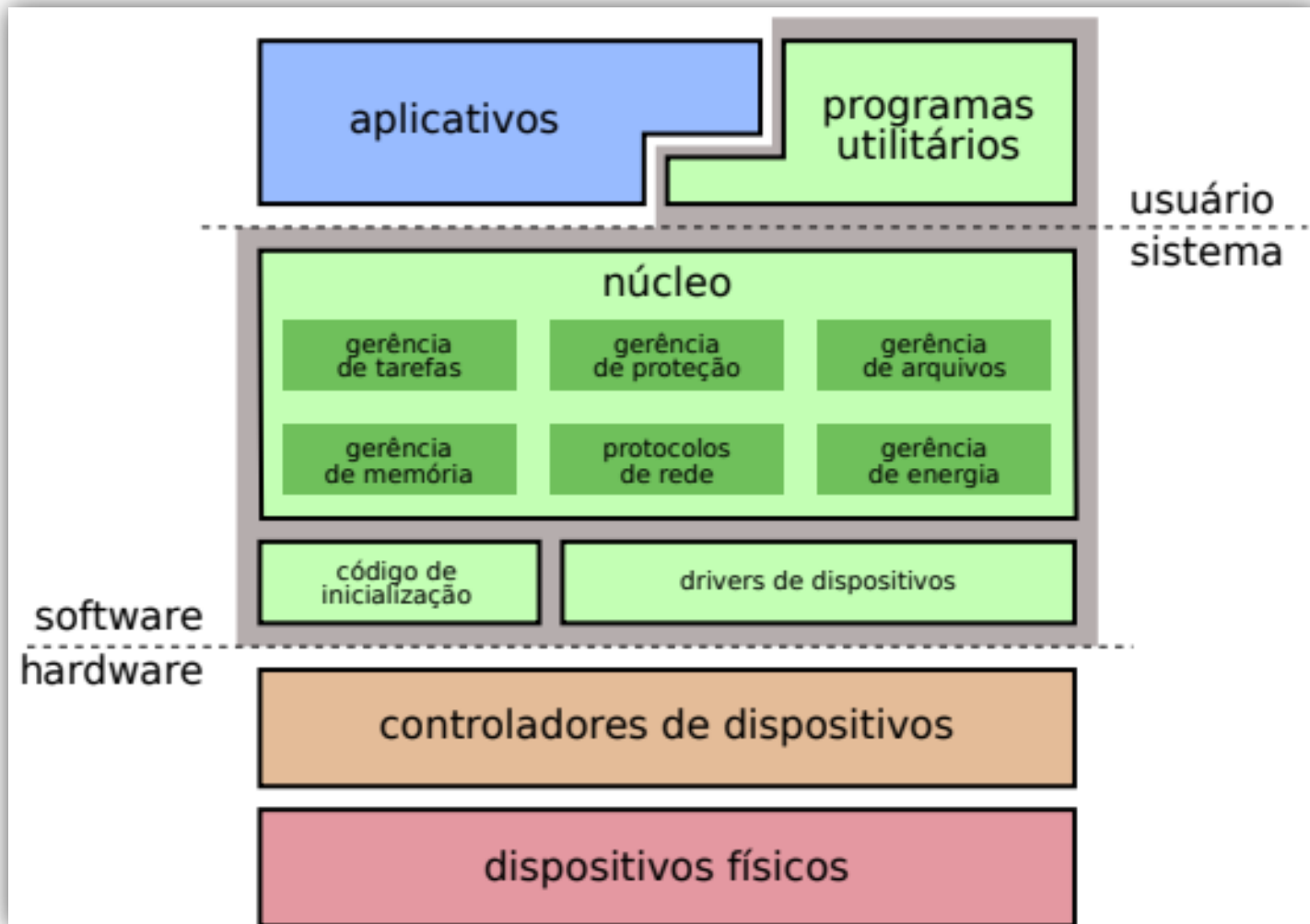
int OP1VAR, OP2VAR, RESVAR;
                        //declara variáveis
int main(void){        //indica início código
    OP1VAR = 5;         //carrega variáveis
    OP2VAR = 12;
    ...
    RESVAR = OP1VAR + OP2VAR;
                        //realiza a soma
    ...
    ...
```



Conceito de Sistemas Operacionais

- O sistema operacional é uma estrutura de software ampla, **muitas vezes complexa**, que incorpora aspectos de baixo nível (como *drivers* de dispositivos e gerência de memória física) e de alto nível (como programas utilitários e a própria interface gráfica).
- **Oferece uma forma homogênea de acesso aos dispositivos físicos.**
- Exigir que o programador domine como funciona cada dispositivo e programar cada detalhe impossibilitaria a construção de software.

Conceito de sistemas operacionais



Estrutura de um sistema operacional típico (Fonte: Maziero)

E...como a vida atual seria sem SO?



O Uso de Sistemas Operacionais trouxe abstração de recursos.

Abstração em TI é uma forma de reduzir a complexidade e tornar o projeto e a implementação mais eficientes em sistemas complexos.

Exemplo: esconder a complexidade técnica de um sistema por trás de uma chamada de API.



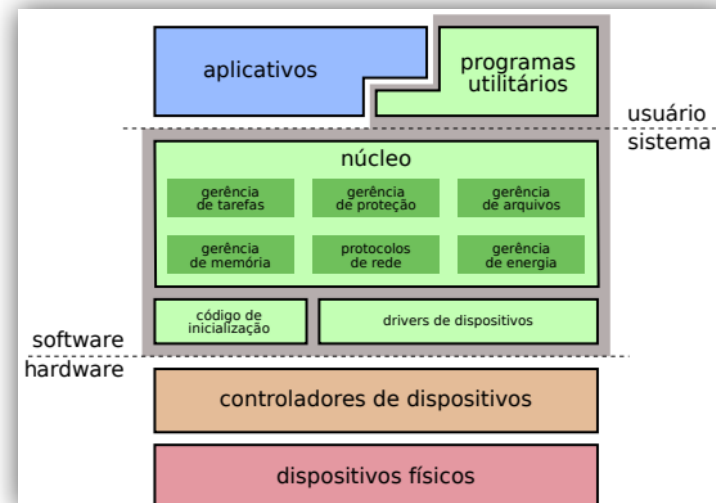
Abstração de Recursos

- Simplifica a construção de programas aplicativos:
- Faz-se uso do conceito de “arquivo” que implementa uma visão abstrata do disco rígido, acessível através de operações como open, read e close.
- Caso tivesse de acessar o disco diretamente, teria de manipular portas de entrada/saída e registradores com comandos para o controlador de disco (sem falar na dificuldade de localizar os dados desejados dentro do disco).

Abstração de Recursos

Tarefa do S.O. definir interfaces abstratas para atender os seguintes objetivos:

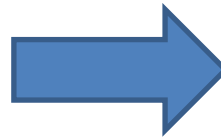
- Prover interfaces de acesso aos dispositivos, mais simples de usar que as interfaces de baixo nível.
- Tornar os aplicativos independentes do hardware.
- Definir interfaces de acesso homogêneas para dispositivos com tecnologias distintas
- Gerenciar Recursos.



SISTEMA OPERACIONAL

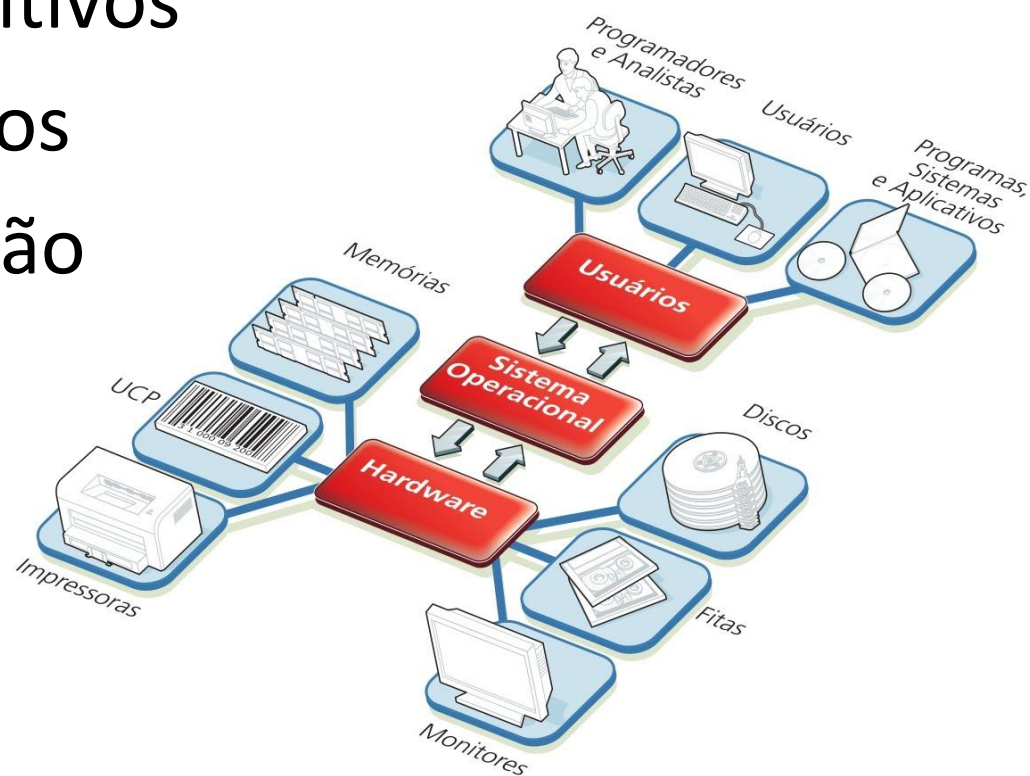
=

O GERENCIADOR DE RECURSOS



Gerenciador de Recursos

- Gerência de processador
- Gerência de memória
- Gerência de dispositivos
- Gerência de arquivos
- Gerência de proteção



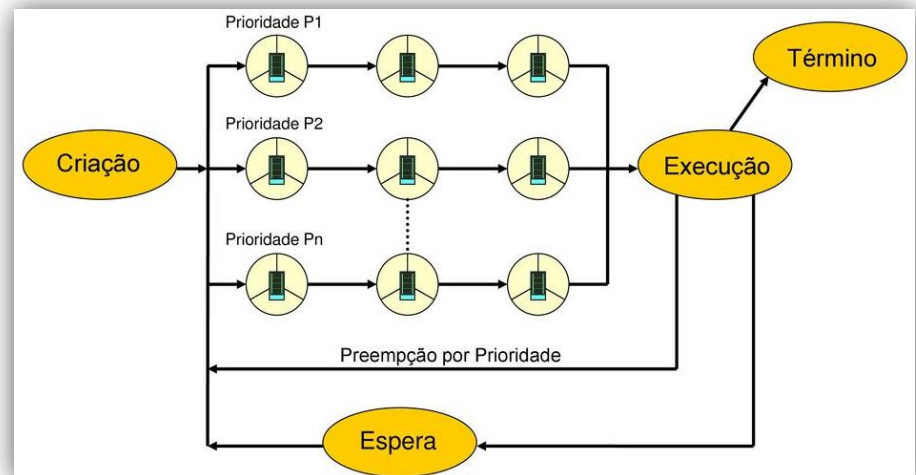
O que é Gerenciar de Recursos?

- **Gerenciar Recursos é uma tarefa do S.O.:**
 - **É controlar e ordenar a alocação dos recursos do computador (memória, dispositivos de E/S, processadores).**
 - **Exemplo de controle:**
 - 03 programas competindo por um impressora: 3 programas pra imprimir:
 - Como garantir que as linhas não saíam misturadas.
 - Proteger a memória de acessos proibidos.
 - Como proteger que os software de um usuário não afete a memória de outro software.

S.O. = Gerenciador de Recursos

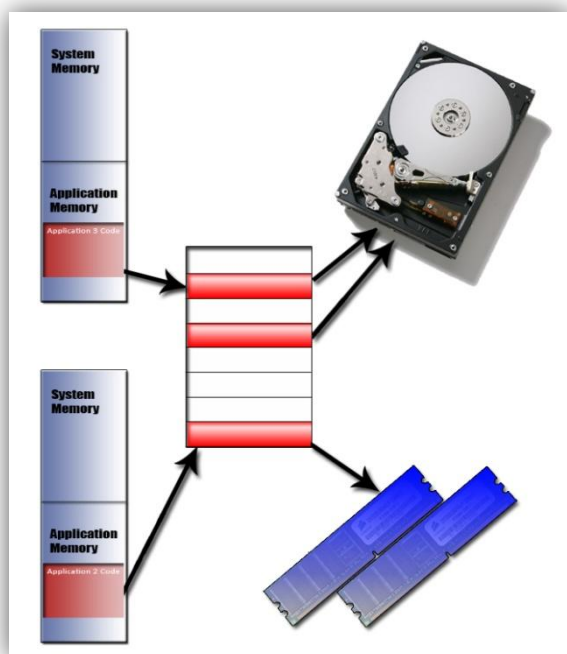
- **Gerência de processador:**

- Também conhecida como gerência de processos ou de atividades, esta funcionalidade visa distribuir a capacidade de processamento de forma justa entre as aplicações, evitando que uma aplicação monopolize esse recurso e respeitando as prioridades dos usuários.



S.O. = Gerenciador de Recursos

- **Gerência de memória:**
 - Tem como objetivo fornecer a cada aplicação uma área de memória própria, independente e isolada das demais aplicações e inclusive do núcleo do sistema.



- O isolamento das áreas de memória das aplicações melhora a estabilidade e segurança do sistema como um todo, pois impede aplicações com erros (ou aplicações maliciosas) de interferir no funcionamento das demais aplicações.

S.O. = Gerenciador de Recursos

- **Gerência de Dispositivos**

- Cada periférico do computador possui suas peculiaridades; assim, o procedimento de interação com uma placa de rede é completamente diferente da interação com um disco rígido SCSI.
- A função da gerência de dispositivos (também conhecida como gerência de entrada/saída) é implementar a interação com cada dispositivo por meio de **drivers** e criar modelos abstratos que permitam agrupar vários dispositivos distintos sob a mesma interface de acesso.

S.O. = Gerenciador de Recursos

- **Gerência de Arquivos**

- Esta funcionalidade é construída sobre a gerência de dispositivos e visa criar arquivos e diretórios, definindo sua interface de acesso e as regras para seu uso.
- É importante observar que os conceitos abstratos de arquivo e diretório são tão importantes e difundidos que muitos sistemas operacionais os usam para permitir o acesso a recursos que nada tem a ver com armazenamento.



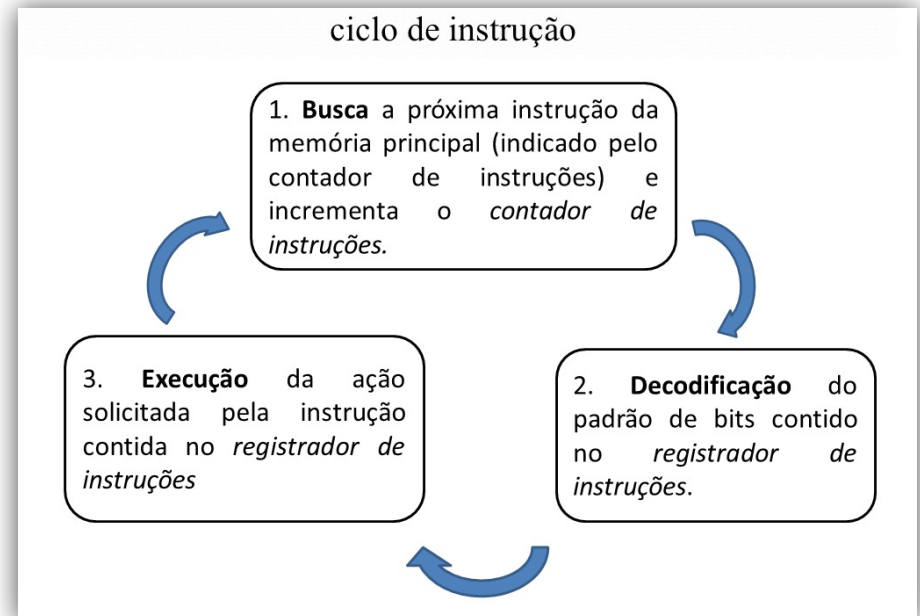
SO = Gerenciador de Recursos

- **Gerência de Proteção**
 - Com computadores conectados em rede e compartilhados por vários usuários, é importante definir claramente os recursos que cada usuário pode acessar, as formas de acesso permitidas (leitura, escrita, etc.) e garantir que essas definições sejam cumpridas.



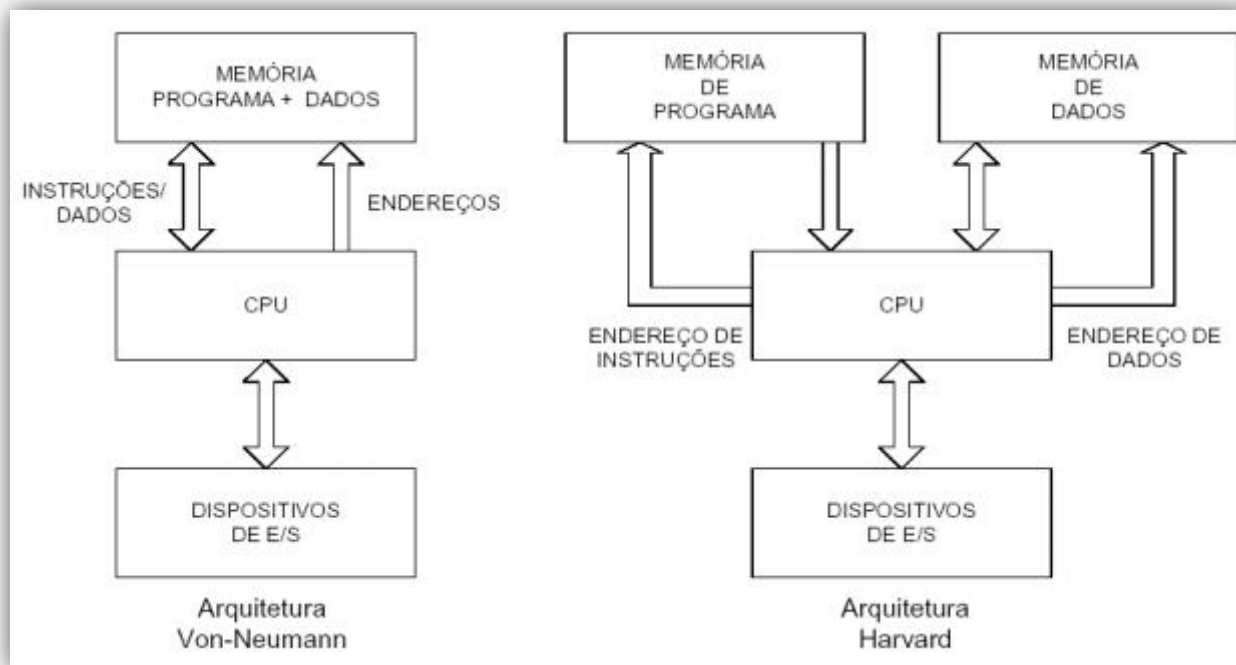
HARDWARE X INTERRUPÇÕES

IMPORTANTE



ARQUITETURA de HARDWARE

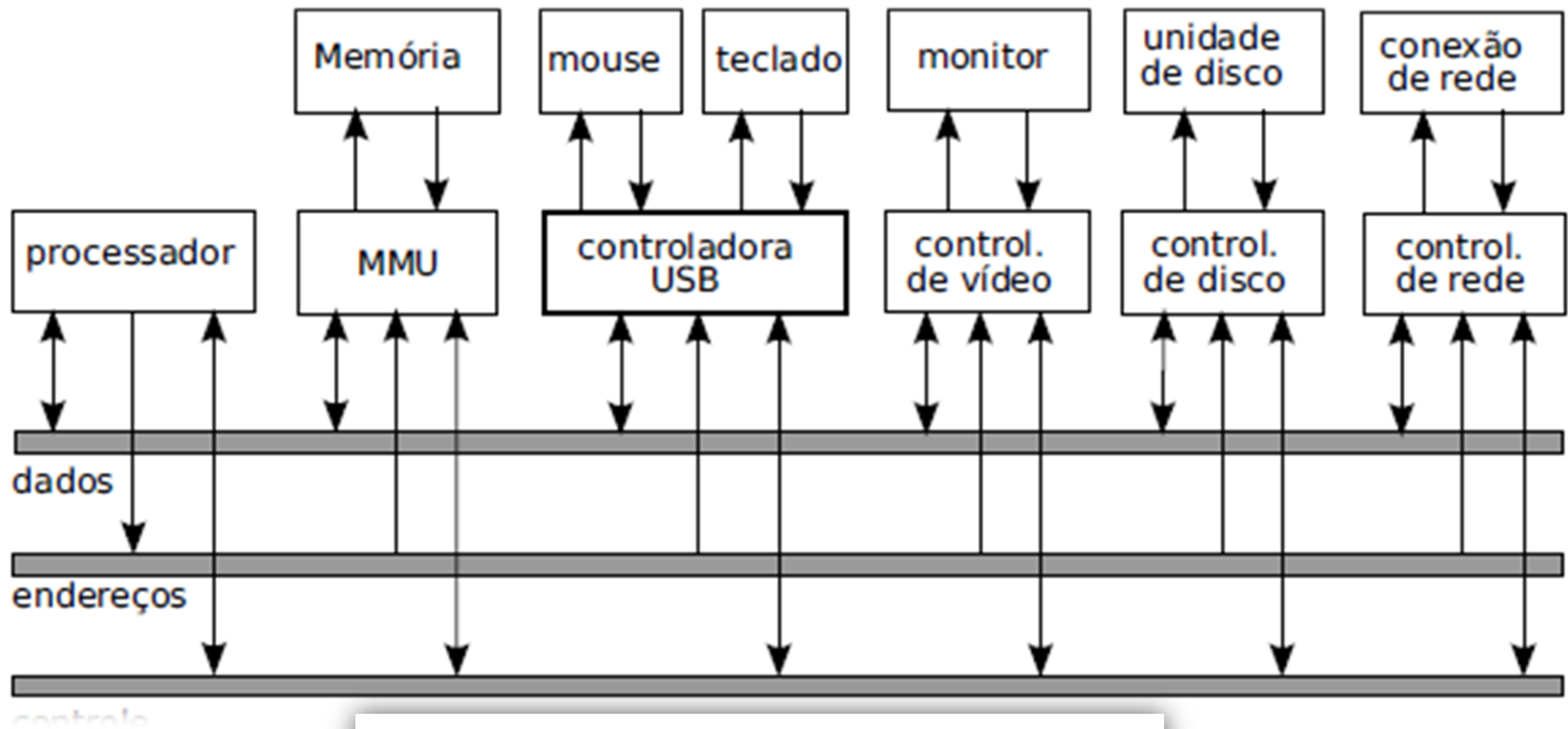
Existem duas arquiteturas clássicas para os microprocessadores em geral: a **arquitetura Von-Neumann**, onde existe apenas um barramento interno por onde circulam instruções e dados e a **arquitetura Harvard**, que é caracterizada por dois barramentos internos, sendo um de instruções e outro de dados.



HARDWARE

- Para o processador, cada dispositivo é representado por seu respectivo controlador **(endereço de hardware)**.
- Os controladores podem ser acessados através de **portas de entrada/saída endereçáveis**: a cada controlador é atribuída uma faixa de endereços de portas de entrada/saída.

HARDWARE



dispositivo	endereços de acesso
teclado	0060h-006Fh
barramento IDE primário	0170h-0177h
barramento IDE secundário	01F0h-01F7h
porta serial COM1	02F8h-02FFh
porta serial COM2	03F8h-03FFh

INTERRUPÇÕES

Imagine o Controlador USB querendo se comunicar com o Processador:

1ª opção: aguarda a consulta do processador



2ª opção: Requisição de interrupção



INTERRUPÇÕES

Aguardar até que o processador o consulte, o que poderá ser **demorado caso o processador esteja ocupado** com outras tarefas (o que geralmente ocorre).

Notificar o processador através do barramento de controle, enviando a ele uma requisição de interrupção (**IRQ – Interrupt ReQuest**).

1. Processador suspende a execução
2. Consulta o vetor de Interrupção (tabela).
3. Desvia para o endereço da Vetor de Interrupção (tabela).
4. Executa a rotina de tratamento de interrupção
5. Retorna a execução anterior.

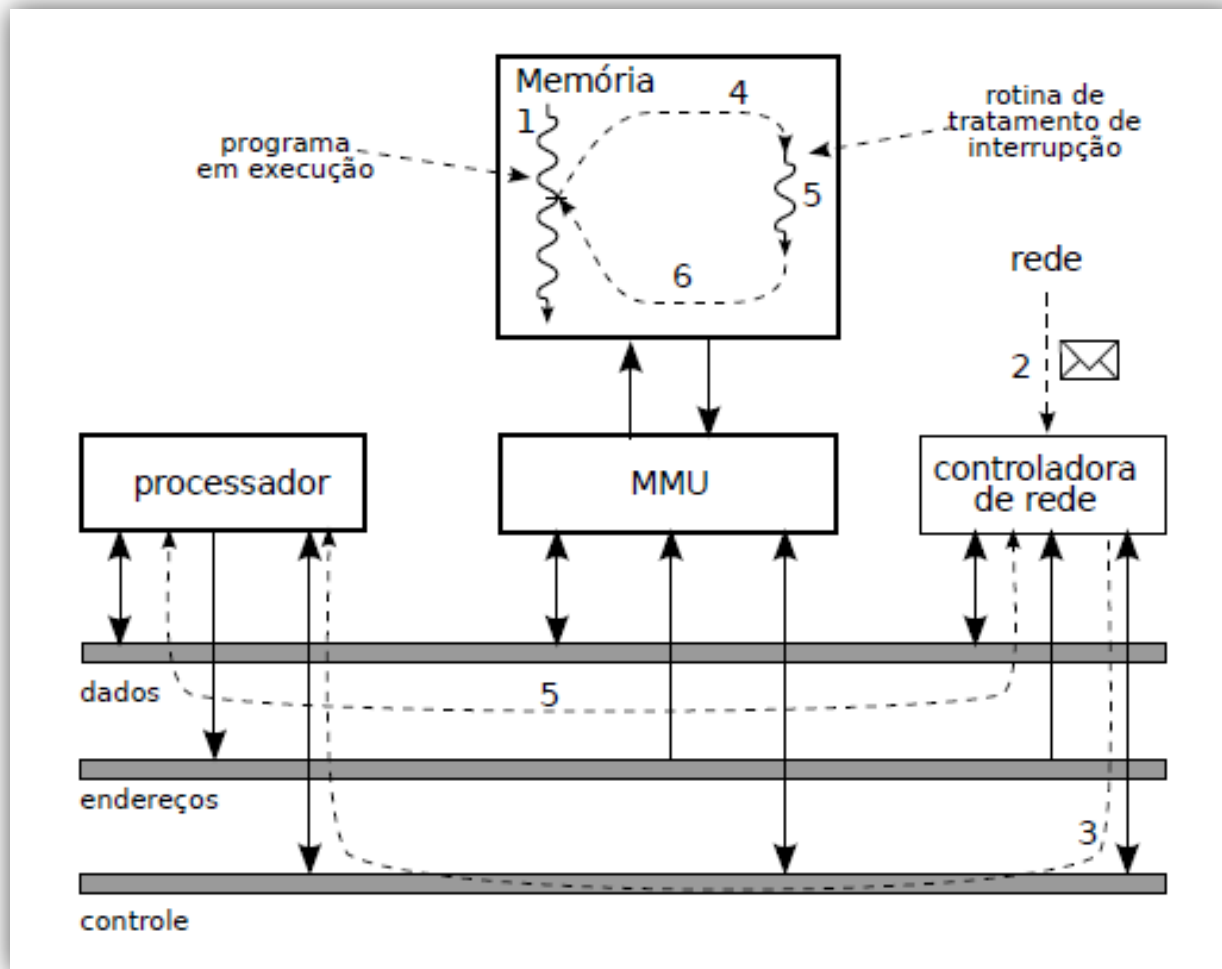
INTERRUPÇÕES

Exemplo:

1. O processador está executando um programa qualquer (em outras palavras, um fluxo de execução);
2. Um pacote vindo da rede é recebido pela placa Ethernet;
3. A placa envia uma solicitação de interrupção (IRQ) ao processador;
4. O processamento é desviado do programa em execução para a rotina de tratamento da interrupção;
5. A rotina de tratamento é executada para receber as informações da placa de rede (via barramentos de dados e de endereços) e atualizar as estruturas de dados do sistema operacional;
6. A rotina de tratamento da interrupção é finalizada e o processador retorna à execução do programa que havia sido interrompido.

INTERRUPÇÕES

Controlador querendo comunicar com o processador.



Proteção do Núcleo

O Sistema Operacional deve gerenciar o hardware:

- Apenas o S.O. deve acessar diretamente o hardware.
- Aplicações pedem ao SO, o qual pedirá ao hardware.

Aplicações com **acesso pleno ao hardware tornariam inúteis** os mecanismos de segurança e controle de acesso aos recursos (tais como arquivos, diretórios e áreas de memória).

Proteção do núcleo



Como se consegue isso?

CPUS tem níveis de privilégio de execução

Flags na CPU que definem o nível de acesso ao hardware.

MODOS DE OPERAÇÃO

NÍVEL NÚCLEO:

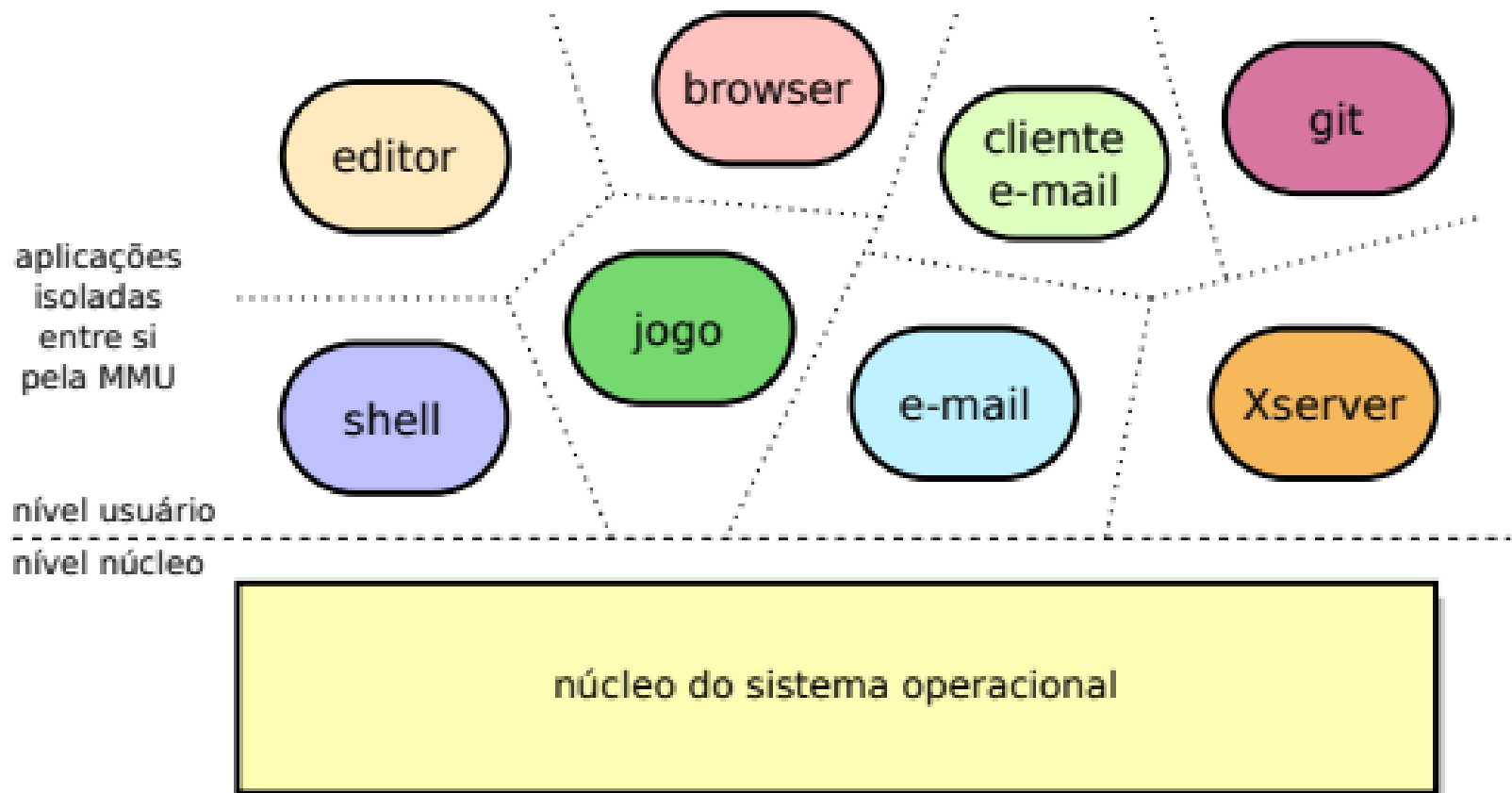
- Também chamado de **nível supervisor, sistema, monitor** ou **kernel space**, esse é o modo com privilégios totais.
- Nele, o código pode acessar todos os recursos do processador, toda a memória e executar qualquer instrução.
- Ao ser ligado, o processador inicia sua execução nesse nível.

MODOS DE OPERAÇÃO

NÍVEL USUÁRIO (ou userspace):

- Neste nível, apenas um subconjunto das instruções do processador, registradores e portas de entrada/saída pode ser utilizado.
- Instruções privilegiadas, como HALT e RESET, são bloqueadas, e o acesso à memória é restrito às áreas autorizadas.
- Caso o programa tente executar uma instrução proibida ou acessar uma região de memória não permitida, o hardware gera uma exceção e transfere o controle ao núcleo do sistema, que normalmente encerra execução do programa.

MODOS DE OPERAÇÃO



Chamadas de Sistemas

**Mas como um programa invoca o S.O. para
acessar um recurso?**



Chamadas de sistemas

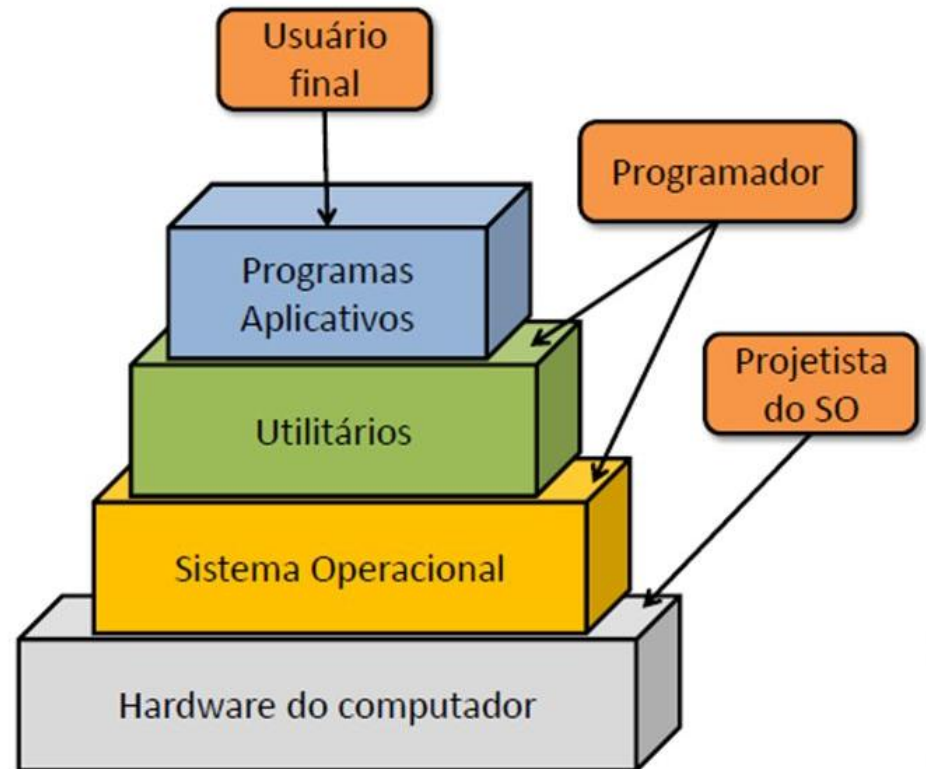
Uma chamada do sistema é uma solicitação feita por um programa ao S.O., ou seja **permite uma aplicação acessar recursos do Sistema Operacional**.

As chamadas do sistema realizam operações no nível do sistema, como comunicação com **Hardware** dispositivos e leitura e escrita arquivos.

Ao fazer chamadas do sistema, os desenvolvedores podem usar funções pré-escritas suportadas pelo sistema operacional (S.O.) em vez de escrevê-las do zero.

Isso simplifica o desenvolvimento, melhora a estabilidade do aplicativo e torna os aplicativos mais "portáteis" entre diferentes versões de um sistema operacional.

ARQUITETURAS DE SISTEMAS OPERACIONAIS



ARQUITETURAS DE SIST. OPERAC.

Um SO pode ter uma arquitetura do tipo:

01) MONOLÍTICOS (ou Monocúcleos)

02) DE CAMADAS

03) MICRONÚCLEO

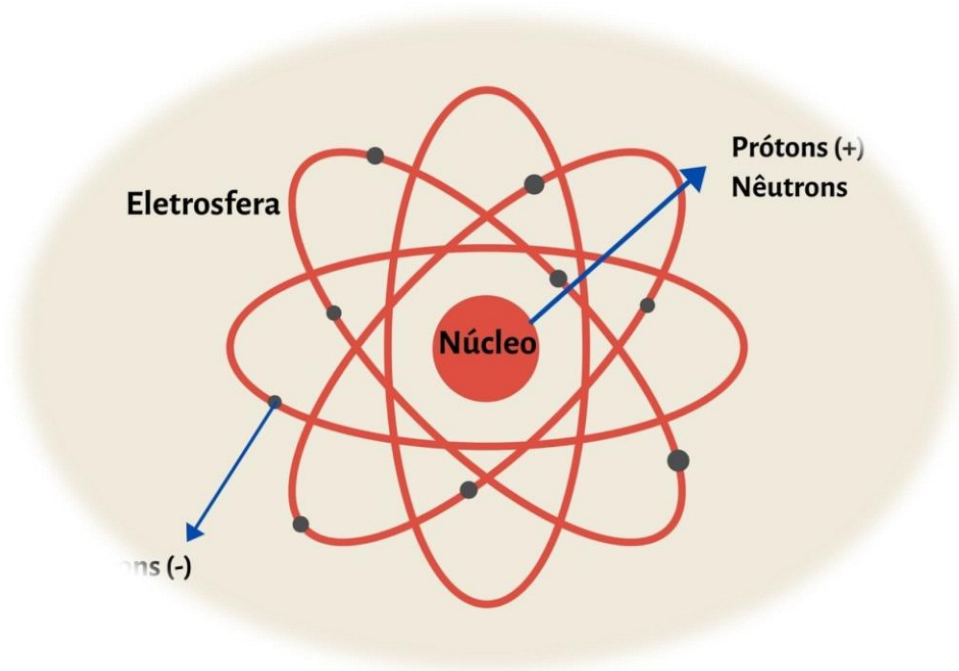
04) SISTEMAS HÍBRIDOS

– MÁQUINAS VIRTUAIS

– EXONÚCLEO

– UNINÚCLEO

– CONTAINERS



01 - SISTEMAS MONILÍTICOS (MONONÚCLEOS)

- Todo o sistema operacional é executado no núcleo
- Opera em modo núcleo, com acesso a todos os recursos do hardware e sem restrições de acesso à memória.
- Grande vantagem da arquitetura monolítica é seu desempenho: qualquer componente do núcleo pode acessar os demais componentes
- Considerando que todos os componentes do S.O. têm acesso privilegiado ao *hardware*, caso um componente perca o controle, pode levar o sistema ao colapso.

Rotina principal: grande programa binário executável. Liga-se as outras rotinas. Expandindo as funcionalidades do S.O.

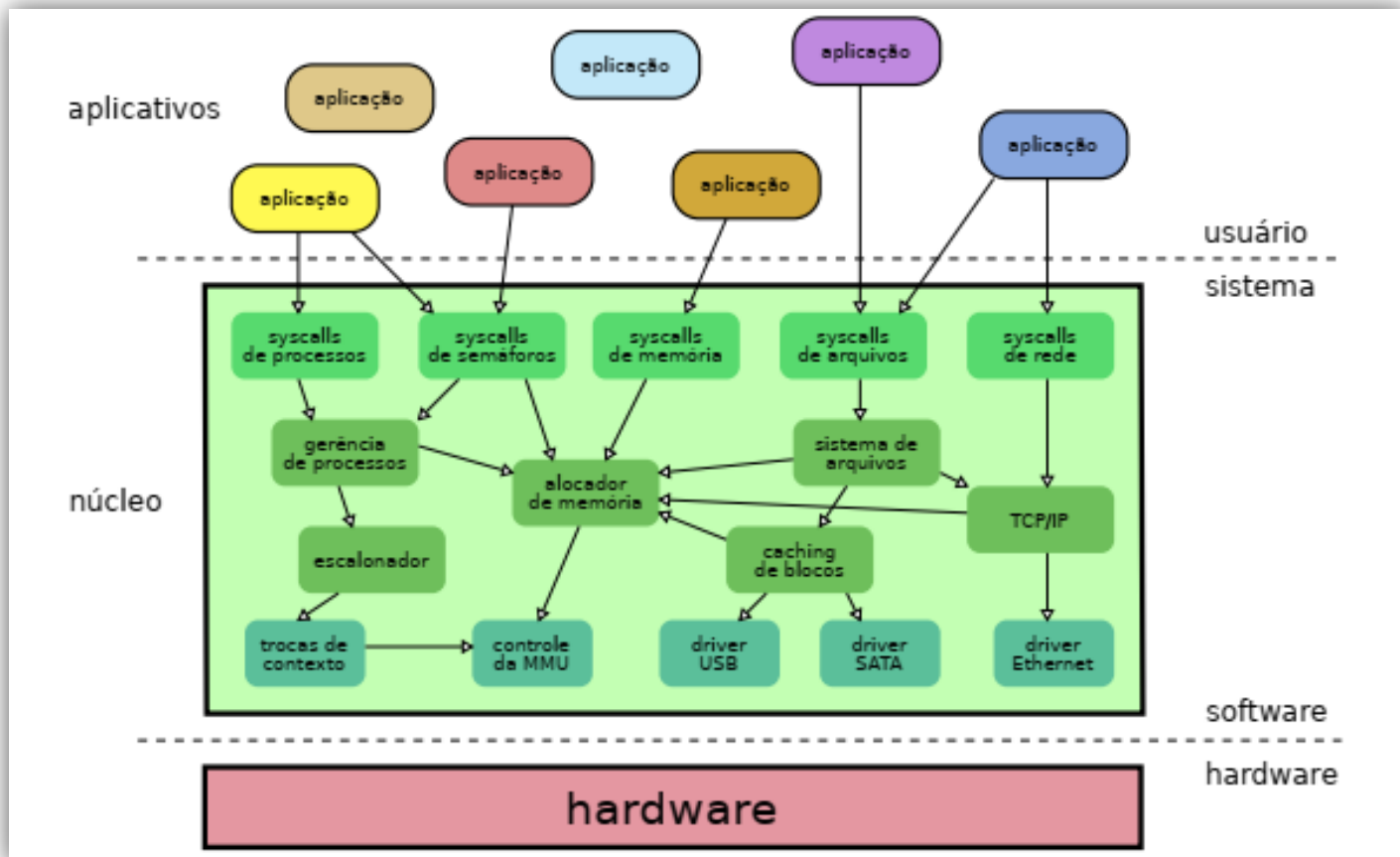
Rotinas de serviços e utilitários: são ligados ao programa binário executável.

Estrutura básica:

- 1) Um programa principal que invoca a rotina do serviço requisitado.
- 2) Um conjunto de rotinas de serviço que executam as chamadas de sistema
- 3) Um conjunto de rotinas utilitárias que auxiliam as rotinas de serviço

01 - SISTEMAS MONILÍTICOS (MONONÚCLEOS)

A arquitetura monolítica foi a primeira forma de organizar os sistemas operacionais; sistemas UNIX antigos e o MS-DOS seguiam esse modelo.



02 - SISTEMAS DE CAMADAS

- Hierarquia de camadas (Divisão).
- A camada mais baixa realiza a interface com o hardware, enquanto as camadas intermediárias proveem níveis de abstração e gerência cada vez mais sofisticados. Por fim, a camada superior define a interface do núcleo para as aplicações (as chamadas de sistema).
- As camadas têm níveis de privilégio decrescentes: a camada inferior tem acesso total ao hardware, enquanto a superior tem acesso bem mais restrito.

02 - SISTEMAS DE CAMADAS

A estruturação em camadas é apenas parcialmente adotada hoje em dia. Como exemplo de sistema fortemente estruturado em camadas pode ser citado o MULTICS e o THE - Technische Hogeschool Eindhoven (1968) e o IBM OS/2

CAMADA	FUNÇÃO
5	O operador
4	Programas de usuário
3	Gerenciamento de entrada/saída
2	Comunicação operador-operador
1	Memória e gerenciamento de tambor
0	Alocação do processador e multiprogramação

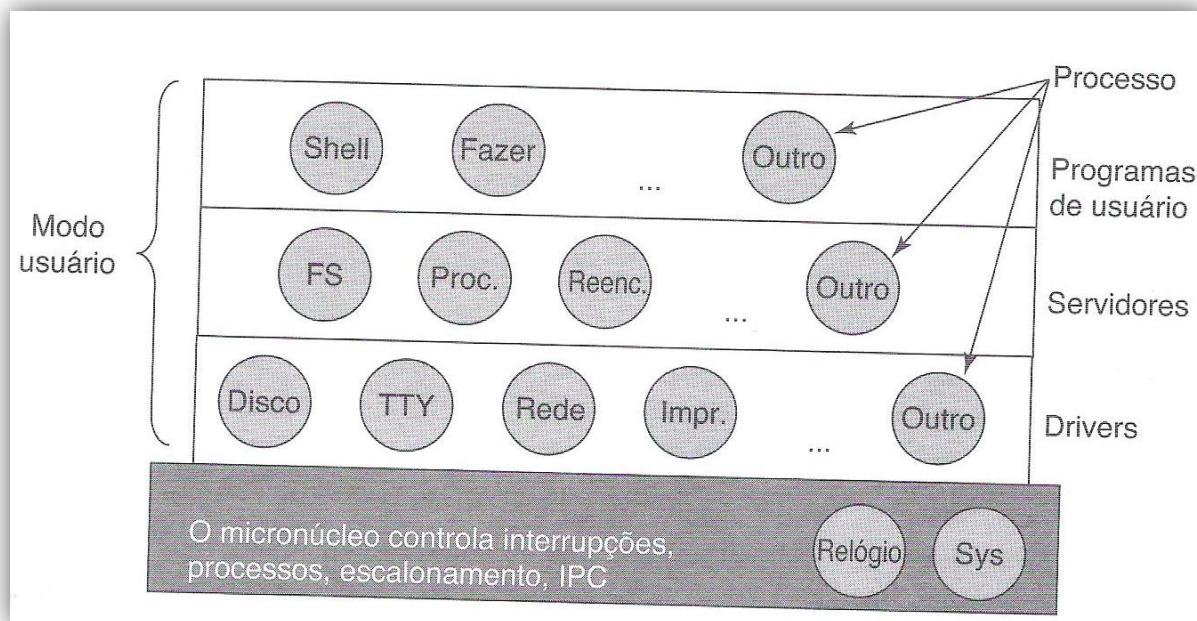
03) SISTEMAS MICRONÚCLEO

- Consistes em dividir o núcleo em pequenos módulos.
- O Objetivo é a diminuição do número de erros no núcleo.
- Consiste em retirar do núcleo todo o código de alto nível, deixando no núcleo somente o código de baixo nível. O restante do código seria transferido para programas separados no espaço de usuário, denominados *serviços*.
- Por fazer o núcleo de sistema ficar menor, essa abordagem foi denominada *micronúcleo* (ou μ -kernel).

Exemplo: Symbian, Minix 3

03) SISTEMAS MICRONÚCLEO

- O micronúcleos vem sendo investigados desde os anos 1980. Os melhores exemplos de micronúcleos bem sucedidos são provavelmente o Minix 3 e o QNX. Vários sistemas operacionais atuais adotam parcialmente essa estruturação, adotando núcleos híbridos, como por exemplo o **MacOS X da Apple**, o **Digital UNIX** e o **Windows NT**.

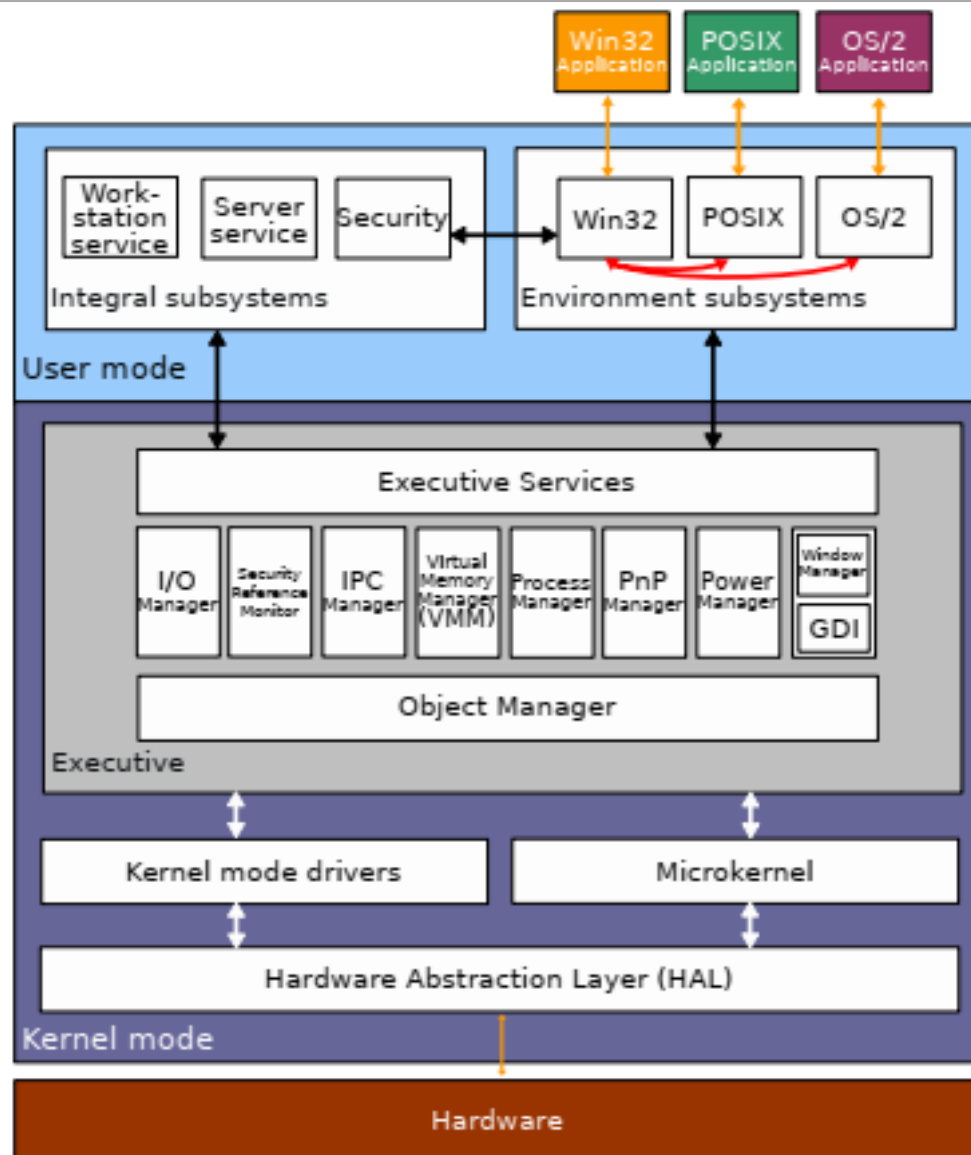


Exemplo: se a unidade de áudio tiver erro, para-se o áudio ou apenas distorce-se o áudio. No monolítico, o sistema inteiro irá parar.

04) SISTEMAS HÍBRIDOS

- Devido ao baixo desempenho dos MICRONÚCLEOS, uma solução encontrada para esse problema consiste em trazer de volta ao núcleo os componentes mais críticos, para obter melhor desempenho.
- Essa abordagem intermediária entre o núcleo monolítico e micronúcleo é denominada *núcleo híbrido*. (também teve a influência da arquitetura em camadas).
- **Vários sistemas operacionais atuais empregam núcleos híbridos, entre eles o Microsoft Windows, a partir do Windows NT.**
- As primeiras versões do Windows NT podiam ser consideradas micronúcleo, mas a partir da versão 4 vários subsistemas foram movidos para dentro do núcleo para melhorar seu desempenho, transformando-o em um núcleo híbrido.
- **Os sistemas MacOS e iOS da Apple também adotam um núcleo híbrido chamado XNU (de *X is Not Unix*), construído a partir dos núcleos Mach (micronúcleo) e FreeBSD (monolítico)**

04) SISTEMAS HÍBRIDOS



Arquiteturas Avançadas

SISTEMAS MÁQUINA VIRTUAIS

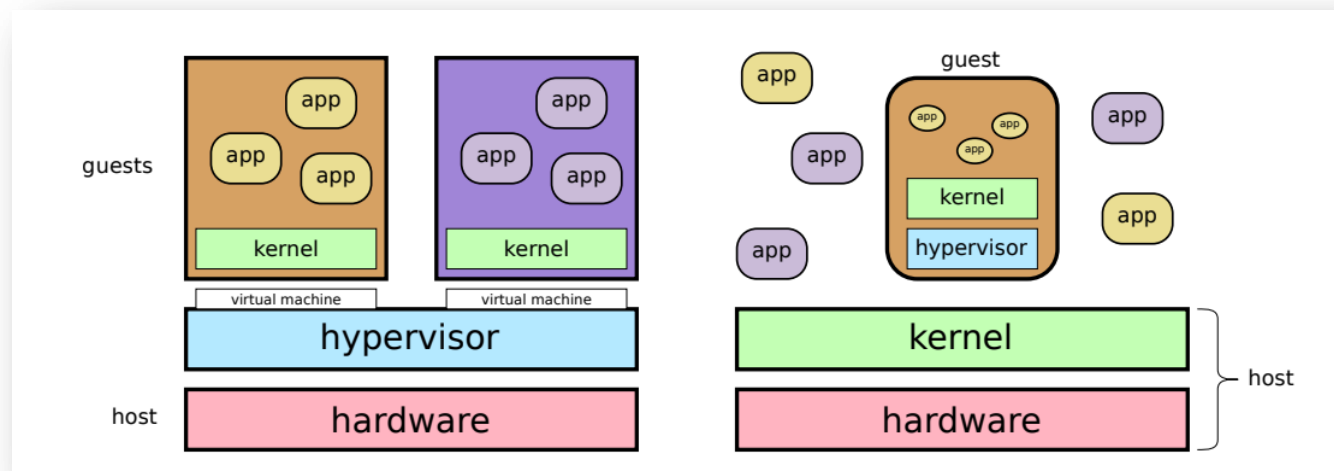
Utilizado em grandes servidores. Ex. IBM z13

Separa os seguintes elementos:

- Multiprogramação
- Estende o hardware com uma interface melhor

Monitor da MV fornece várias máquinas virtuais.

Cada MV pode rodar um S.O. diferente.

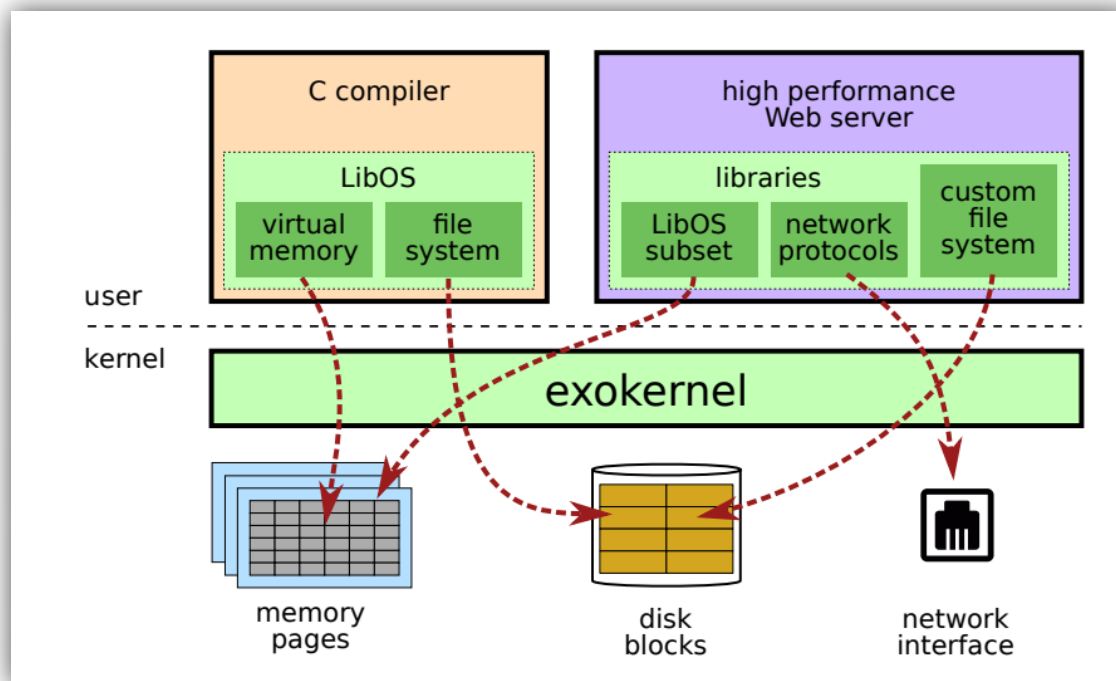


SISTEMAS EXONÚCLEO

- Os exonúcleos (*exokernels*) o núcleo do sistema apenas proporciona acesso controlado aos recursos do *hardware*, mas não implementa nenhuma abstração.
- Todas as abstrações e funcionalidades de gerência que uma aplicação precisa terão de ser implementadas pela própria aplicação, em seu espaço de memória
- Subdivide a máquina real: Um pouco de cada recurso para cada usuário.
- Não precisa camada de mapeamento.

SISTEMAS EXONÚCLEO

- É importante observar que um exonúcleo é diferente de um micronúcleo, pois neste último as abstrações são implementada por um conjunto de processos servidores independentes e isolados entre si, enquanto no **exonúcleo cada aplicação implementa suas próprias abstrações** (ou as incorpora de bibliotecas compartilhadas).
- Até o momento não existem exonúcleos em produtos comerciais



SISTEMAS UNINÚCLEO

- Nos sistemas uninúcleo (*unikernel*) as bibliotecas e uma aplicação são compilados e ligados entre si, formando um bloco monolítico de código.
- O custo da transição aplicação / núcleo nas chamadas de sistema diminui muito, gerando um forte ganho de desempenho.
- Adequada para computadores que executam uma única aplicação, como servidores de rede (HTTP, DNS, DHCP).
- Sistemas embarcados e nas chamadas ***appliances*** de rede (roteadores, modems, etc), é usual compilar o núcleo, bibliotecas e aplicação em um único arquivo binário que é em seguida executado diretamente sobre o hardware, em modo privilegiado.

SISTEMAS CONTAINERS

- O espaço de usuário do sistema operacional é dividido em áreas isoladas denominadas *domínios* ou *contêineres*.
- A cada domínio é alocada uma parcela dos recursos do sistema operacional, como memória, tempo de processador e espaço em disco.
- Para garantir o isolamento entre os domínios, cada domínio tem sua própria interface de rede virtual e, portanto, seu próprio endereço de rede (IP).
- Processos não podem trocar de domínio nem acessar recursos de outros domínios. Os domínios são vistos como sistemas distintos, acessíveis somente através de seus endereços de rede.

SISTEMAS CONTAINERS

- Além disso, na maioria das implementações cada domínio tem seu próprio espaço de nomes para os identificadores de usuários, processos e primitivas de comunicação.
- O núcleo do sistema operacional é o mesmo para todos os domínios.
- Processos em um mesmo domínio podem interagir entre si normalmente, mas processos em domínios distintos não podem ver ou interagir entre si.
- Processos não podem trocar de domínio nem acessar recursos de outros domínios. Os domínios são vistos como sistemas distintos, acessíveis somente através de seus endereços de rede.
- O núcleo Linux oferece diversos mecanismos para o isolamento de espaços de recursos, que são usados por plataformas de gerenciamento de contêineres como *Linux Containers (LXC)*, *Docker* e *Kubernetes*.

SISTEMAS CONTAINERS

